

Lab 4 - Laser Scanner

After implementing the tasks in the previous labs, you have the knowledge to conduct:

- creating and building ROS2 package,
- launching package with `ros2 run` or `ros2 launch` commands,
- using ROS tools to examine nodes and topics,
- sending velocity commands to a *turtlebot*, and
- generating the robot trajectory and velocity graphs.

In this lab, you will use the laser sensor data to control the robot moving from the home, through two gaps and reach the charger position. The original odometry based control code need to be removed so that the laser based new control code can be created. **Note that you should keep a copy of your code `tutorial_pkg_node.cpp` written in Lab 4 for backup in case you may need it in the future, i.e.**

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg/src
cp tutorial_pkg_node.cpp tutorial_pkg_Lab5.cpp
```

Task 1 Based on the file `tutorial_pkg_node.cpp` created in the last Lab, you should add the laser commands into your control code to collect laser range readings (*Range, theta*) at five angles and stored them into the .csv file for viewing and plotting.

At this stage, the five laser range readings are selected for your learning process. You may use more laser range readings in your assignment in the next stage.

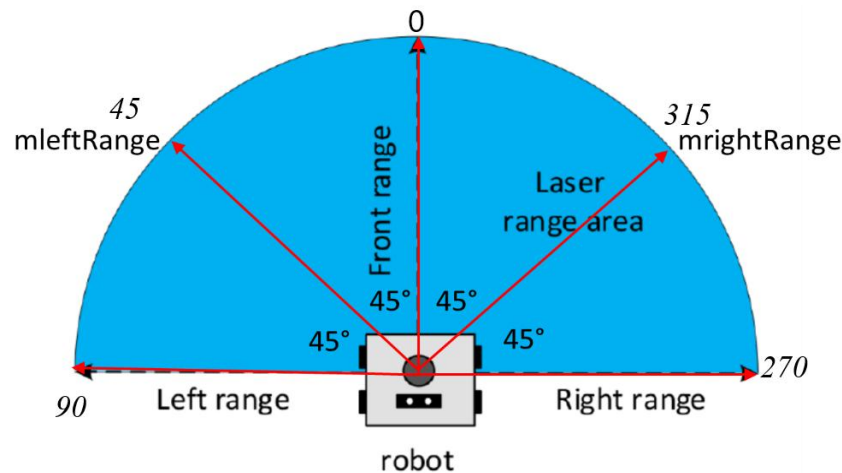


Figure 6.1 The laser sensor that takes 5 range readings (*Range, theta*) (*leftRange, 90°*), (*mleftRange, 45°*) (*frontRange, 0*), (*rightRange, 270°*), (*mrightRange, 315°*).

Step 1: Using the following commands to open `tutorial_pkg_node.cpp`

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg/src
gedit tutorial_pkg_node.cpp
```

Then add the Yellow-highlighted codes below.

```
#include<chrono>
```

```

#include<functional>

// existing code

#include "nav_msgs/msg/odometry.hpp"

#include "sensor_msgs/msg/laser_scan.hpp"

std::ofstream robotTrajFile;
std::ofstream robotVelFile;
std::ofstream robotLaserFile; // Declare a file object for recording your laser data.

```

Step 2: You need to subscribe to the laser topic ‘/scan’ in

```

class Stopper:public rclcpp::Node{
public:
    // existing code.

    /* class constructor */
    Stopper():Node("Stopper"), count_(0){

    // existing code.

    laserScan_ = this->create_subscription<sensor_msgs::msg::LaserScan>("scan",
    10, std::bind(&Stopper::scanCallback, this, std::placeholders::_1));
    };

```

Step 3: You need the following Yellow codes into Stopper class as well:

```

class Stopper:public rclcpp::Node {
public:
    // some existing parameters.

    double robVelocity;
    double frontRange, mleftRange, leftRange, rightRange, mrightRange;
    int laser_index = 0; // index the laser scan data
    /* moving function */
    void scanCallback(const sensor_msgs::msg::LaserScan::SharedPtr scan);
Private:
    // some existing parameters.

    rclcpp::Subscription<sensor_msgs::msg::LaserScan>::SharedPtr laserScan_;
}

```

Step 4: to add the ‘scanCallback’ function after “odomCallback()” function.

```

void Stopper::scanCallback(const sensor_msgs::msg::LaserScan::SharedPtr scan){
    leftRange = scan->ranges[90]; // get a range reading at the left angle
    mleftRange = scan->ranges[45]; // get a range reading at the mid-left angle
    frontRange = scan->ranges[0]; // get a range reading at the front angle
    mrightRange = scan->ranges[315]; // get a range reading at the mid-right angle
    rightRange = scan->ranges[270]; // get the range reading at the right angle

```

```
robotLaserFile << leftRange << "," << mleftRange << "," << frontRange << "," <<
mrightRange << "," << rightRange << "," << laser_index++ << endl; }
```

Step 5: Finally, you need to add the following code into “*void Stopper::startMoving()*”.

Use the path you previously used for *robotTrajData* and *robotVelData*, to declare where *robotLaserData.csv* should be created. Once that is done, save the file and exit.

```
void Stopper::startMoving(){
    // existing code

    robotLaserFile.open("~/M-Drive/ros2_ws/src/tutorial_pkg/laserData.csv",ios::trunc);
    // existing code
    while (rclcpp::ok()){ // keep spinning loop until user presses Ctrl+C
        rclcpp::WallRate loop_rate(20);
        auto node = std::make_shared<Stopper>();
        rclcpp::spin(node);
        loop_rate.sleep();
    }
    // existing code
    robotLaserFile.close();
}
```

Then, save the file and exit from it.

Step 6: Now you need to revise *packagr.xml* using the commands below.

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg
gedit package.xml
```

you should add the following line below the line *<depend>nav_msgs</depend>*

```
<depend>sensor_msgs</depend>
```

Save the file.

Use the following command to open the file *CMakeLists.txt*.

```
gedit CMakeLists.txt
```

Add the following lines below the line *find_package(nav_msgs REQUIRED)*

```
find_package(sensor_msgs REQUIRED)
```

Then add *sensor_msgs* into the following line:

```
ament_target_dependencies(tutorial_pkg_node rclcpp std_msgs geometry_msgs
nav_msgs sensor_msgs)
```

Then, save the file and exit from it.

Step 7: Now you can compile and run your code by using the commands below.

```
cd ~/M-Drive/ros2_ws
source install/setup.sh
export TURTLEBOT3_MODEL=burger
```

colcon build

ros2 launch tutorial_pkg tutorial_turtlebot.launch.py

Step 8: You may use LibreOffice in Ubuntu or Excel to view the laser data in the csv file.

Polar to Cartesian for building a dynamic environment map

Figure 6.2 shows the laser readings with Polar coordinates, $(Range, \theta)$, in a sensor frame that is centred at the robot local frame. However, the robot location uses Cartesian coordinates, (X, Y) in a global frame. To build a dynamic environment map, you should translate your laser readings from Polar coordinates, $(Range, \theta)$, in the robot local frame to Cartesian coordinates (x_g, y_g) in a global frame for the robot navigation and obstacle avoidance.

As can be seen in Figure 6.2, (X, Y) is the robot's position in the world coordinate system.

(x_r, y_r) is the obstacle's position in the robot coordinate system. (x_g, y_g) is the detected object position in the global coordinate system, which can be calculated by using

$$x_g = Range * \cos(\theta + Th) + X \quad (1)$$

$$y_g = Range * \sin(\theta + Th) + Y \quad (2)$$

where Th is the heading angle of the robot in the global coordinate system. θ is the angle between the laser range and the robot's heading direction.

It should be noticed that the heading angle of the robot, Th , is not available from the Gazebo simulator. You need calculate it by using the orientation (x, y, z, w) of the robot that is available in the message published from the '/odom' topic.

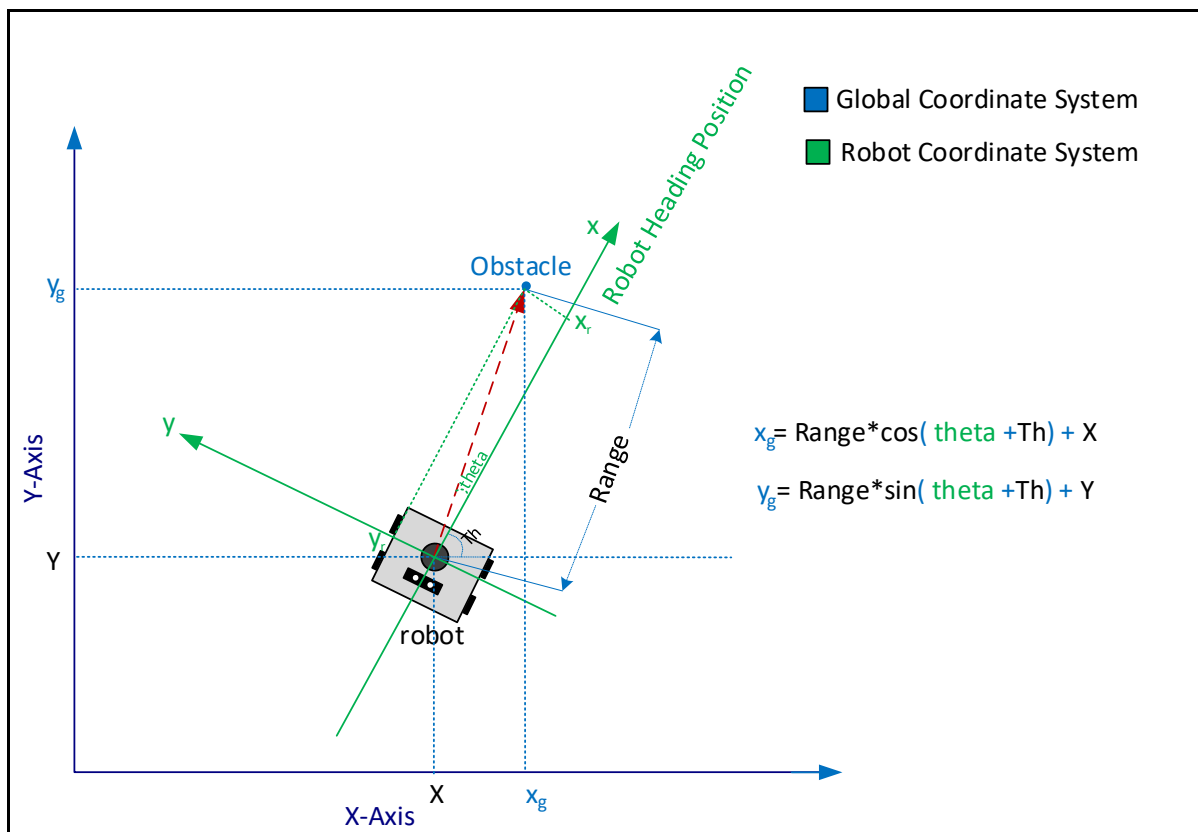


Figure 6.2 From laser range to world coordinate.

After the head angle of the robot, Th , is obtained, you can use it to do the calculations of equations (1) and (2) to obtain the Cartesian coordinates of the laser range, (x_g, y_g) , each time you have obtained a laser reading data.

Task 2: To add the new code into `tutorial_pkg_node.cpp` so that you can obtain the head angle of the robot, Th , and then use it to do the calculations of equations (1) and (2) to obtain the Cartesian coordinates of the laser range, (x_g, y_g) each time a laser reading data is obtained. Finally, you should store the calculated the Cartesian coordinates of the laser readings into the csv file for viewing and plotting.

Step 1: To include the following code at the beginning of `tutorial_pkg_node.cpp`:

```
// existing code
std::ofstream laserMapFile;
struct EulerAngles{double roll, pitch, yaw;}; // yaw is what you want, i.e. Th
struct Quaternion{double w, x, y, z;};

EulerAngles ToEulerAngles(Quaternion q){ // for calculating Th
    EulerAngles angles;
    // roll (x-axis rotation)
    double sinr_cosp = +2.0 * (q.w * q.x + q.y * q.z);
    double cosr_cosp = +1.0 - 2.0 * (q.x * q.x + q.y * q.y);
    angles.roll = atan2(sinr_cosp, cosr_cosp);
    // pitch (y-axis rotation)
    double sinp = +2.0 * (q.w * q.y - q.z * q.x);
    if (fabs(sinp) >= 1)
        angles.pitch = copysign(M_PI/2, sinp); //use 90 degrees if out of range
    else
        angles.pitch = asin(sinp);
    // yaw (z-axis rotation)
    double siny_cosp = +2.0 * (q.w * q.z + q.x * q.y);
    double cosy_cosp = +1.0 - 2.0 * (q.y * q.y + q.z * q.z);
    angles.yaw = atan2(siny_cosp, cosy_cosp);
    return angles;
}
```

Step 2: To add the following code into `'Class Stopper{ }'`

```
class Stopper : public rclcpp::Node {
public:
    // existing parameters.
    Quaternion robotQuat;
    EulerAngles robotAngles;
    double robotHeadAngle;
```

```

double leftAngle=M_PI/2, mleftAngle=M_PI/4, frontAngle=0;
double mrightAngle=-M_PI/4, rightAngle=-M_PI/2;
void transformMapPoint(ofstream& fp, double laserRange, double laserTh,
double robotTh, double robotX, double robotY);

Private:
    // existing parameters.
}

```

Step 3: to add the following code into the function *odomCallback*:

```

void Stopper::odomCallback(const nav_msgs::msg::Odometry::SharedPtr odomMsg)
{
    // existing code .....
    rebotVelFile << numberOfCycle++ << ", " << robVelocity << endl;

    robotQuat.x = odomMsg->pose.pose.orientation.x;
    robotQuat.y = odomMsg->pose.pose.orientation.y;
    robotQuat.z = odomMsg->pose.pose.orientation.z;
    robotQuat.w = odomMsg->pose.pose.orientation.w;
    robotAngles = ToEulerAngles(robotQuat);
    robotHeadAngle = robotAngles.yaw;
    switch(stage){
    // existing code ....
}

```

Step 4: Then add the following yellow-highlighted code into *scanCallback()*

```

void Stopper::scanCallback(const sensor_msgs::msg::LaserScan::SharedPtr scan)
{
    // existing code
    transformMapPoint(laserMapFile, frontRange, frontAngle, robotHeadAngle, PositionX, PositionY);
    transformMapPoint(laserMapFile, mleftRange, mleftAngle, robotHeadAngle, PositionX, PositionY);
    transformMapPoint(laserMapFile, leftRange, leftAngle, robotHeadAngle, PositionX, PositionY);
    transformMapPoint(laserMapFile, rightRange, rightAngle, robotHeadAngle, PositionX, PositionY);
    transformMapPoint(laserMapFile, mrightRange, mrightAngle, robotHeadAngle, PositionX, PositionY);
}

```

Step 5: To add the following function after *void Stopper::scanCallback()*

The function aims to transform the laser points from sensor frame to global frame.

```
void Stopper::transformMapPoint(ofstream& fp, double laserRange, double laserTh,
double robotTh, double robotX, double robotY)
{
    double transX, transY;
    transX = laserRange * cos(robotTh + laserTh) + robotX;
    transY = laserRange * sin(robotTh + laserTh) + robotY;
    if (transX < 0) transX = 0;
    if (transY < 0) transY = 0;
    fp << transX << ", " << transY << endl;
}
```

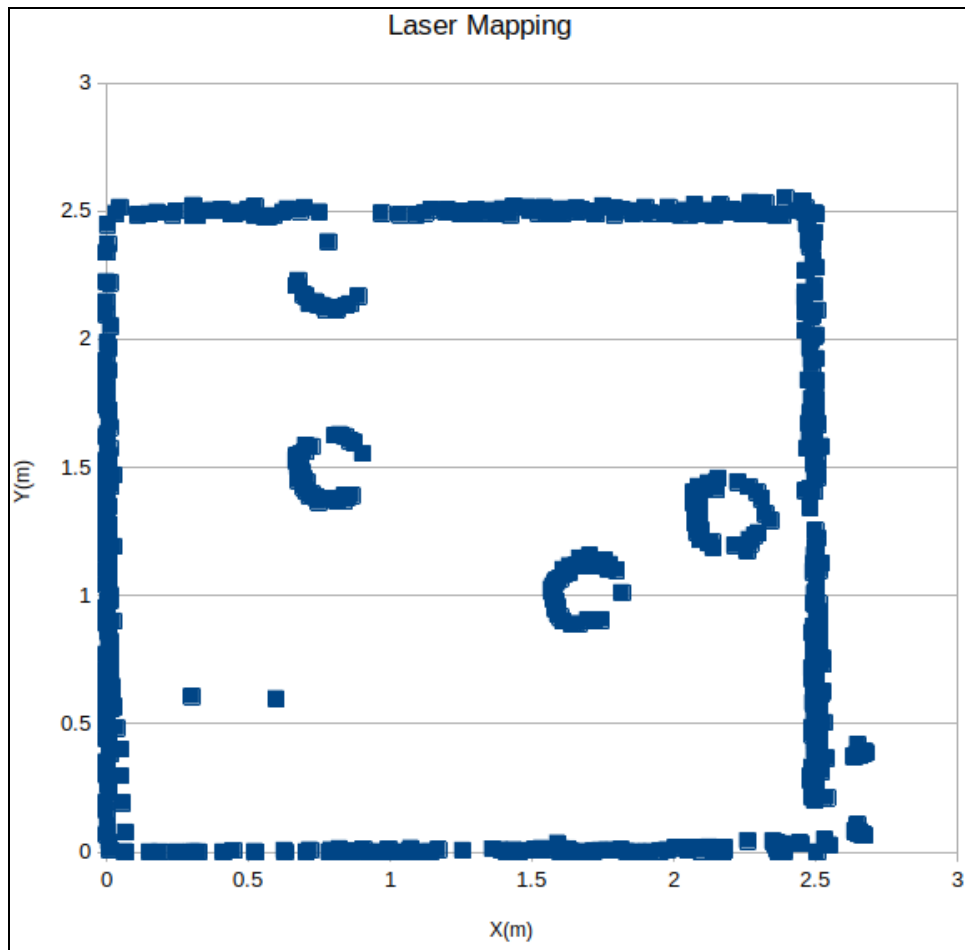
Step 6: Finally, you need to add the following code into “*void Stopper::startMoving()*”.
// Note that you should use your login name to replace “*hhu*” in the path below.

```
void Stopper::startMoving(){
    // some existing code
    laserMapFile.open("/ufs/servh01/users/hhu/M-
Drive/ros2_ws/src/tutorial_pkg/laserMapData.csv",ios::trunc);
    RCLCPP_INFO(this->get_logger(), "Start moving");
    while (rclcpp::ok()){ // keep spinning loop until you press Ctrl+C
        rclcpp::WallRate loop_rate(20);
        auto node = std::make_shared<Stopper>();
        rclcpp::spin(node); // ros::spin(node);
        loop_rate.sleep();
    }
    robotTrajFile.close();
    robotVelFile.close();
    laserData.close();
    laserMapFile.close();
}
```

Step 7: Now you can compile and run your code by using the commands below.

```
cd ~/M-Drive/ros2_ws
source install/setup.sh
export TURTLEBOT3_MODEL=burger
colcon build
ros2 launch tutorial_pkg tutorial_turtlebot.launch.py
```

Step 8: Using LibreOffice in Ubuntu to view the laser map data in the csv file. You may optimize your code to improve the quality of the map if you have time.



Reference:

- [1] Wikipedia, Conversion between quaternions and Euler angles (Accessed 11/12/2021). https://en.wikipedia.org/wiki/Conversion_between_quaternions_and_Euler_angles.